Registro de fuentes y uso responsable de IA

Adjunta este registro a tus tareas cuando utilices herramientas de IA.

Actividad / Código: AD01 - Aplicaciones Distribuidas

Estudiante: [Tu Nombre y Apellido Aquí]

Estudiante: Gabriel Lozano

Declaración de uso:

☐ No utilicé herramientas de IA generativa. Las referencias incluidas fueron consultadas directamente.

☑ Utilicé IA generativa como apoyo. Documentaré a continuación los prompts relevantes y verificaré las referencias antes de citarlas.

Registro de Consultas

Herramienta / modelo	Google Gemini
Fecha	05 de noviembre de 2026
Propósito de la consulta	Ampliar conocimientos sobre cómo estructurar la conexión de tres bases de datos distintas simultáneamente en un entorno de microservicios con ASP.NET Core, manteniendo su independencia.
Prompt exacto	"Hola Gemini, estoy haciendo un proyecto con ASP.NET Core donde tengo 3 microservicios (Catálogo, Inventario y Pedidos) y cada uno tiene su propia base de datos en SQL Server. ¿Cómo hago para que interactúen o se comuniquen entre ellos si necesito datos de los tres al mismo tiempo? ¿Dónde y cómo configuro las conexiones de forma correcta para que sigan siendo independientes y no se mezclen?"
Respuesta aprovechada	La IA explicó el patrón "Database-per-service", aclarando que cada microservicio debe tener su propia cadena de conexión en su archivo `appsettings.json` y no debe consultar bases de datos ajenas directamente. Para interactuar al mismo tiempo, sugirió utilizar llamadas HTTP asíncronas (HttpClient) entre los microservicios para solicitar los datos necesarios sin romper el acoplamiento débil.
Fuentes informales y sugeridas por IA (al menos 3)	1. Documentación de Microsoft Learn sobre arquitectura de microservicios. 2. Artículos sobre el patrón Database-per-service en Microservices.io. 3. Foros de StackOverflow sobre configuración de `DbContext` múltiple.
Fuentes verificadas y citadas (al menos 3)	1. Microsoft Learn (s.f.) 2. Richardson (s.f.) 3. Microsoft Learn - EF Core (s.f.)

Herramienta / modelo	Google Gemini
Fecha	05 de noviembre de 2026
Propósito de la consulta	Comprender cómo implementar validaciones adecuadas en los endpoints de las Web APIs para retornar los códigos de estado HTTP correctos (ej. 200, 400, 404).
Prompt exacto	"En mis APIs de ASP.NET Core, ¿cómo puedo hacer validaciones correctas al momento de consultar, crear o modificar un registro para devolver los mensajes HTTP adecuados? Es decir, devolver un 200 si todo sale bien, un 404 si el registro de la base no existe, o un 400 si hay un error en los datos que se enviaron en el Postman."
Respuesta aprovechada	La IA proporcionó ejemplos del uso de la clase `ControllerBase`, detallando el uso de métodos nativos como `Ok()` para respuestas 200, `BadRequest()` (400) para errores de validación de modelo o entrada incorrecta, y `NotFound()` (404) para consultas que retornan nulo desde la base de datos. Se aplicaron estos conceptos en los controladores de la Práctica 1.
Fuentes informales y sugeridas por IA (al menos 3)	1. Guía de ASP.NET Core sobre códigos de estado de acción HTTP. 2. Tutoriales de validación de Modelos en Web API de C#. 3. Buenas prácticas de diseño de APIs RESTful en Mozilla Developer Network.
Fuentes verificadas y citadas (al menos 3)	1. Microsoft Learn - HTTP (s.f.) 2. Microsoft Learn - Validación (s.f.) 3. MDN Web Docs (s.f.)

Bibliografía / referencias verificadas

- Mozilla Developer Network (s.f.). *Códigos de estado de respuesta HTTP*. MDN Web Docs. Recuperado de la documentación oficial web.
- Microsoft. (s.f.). *Diseño de una arquitectura orientada a microservicios*. Microsoft Learn. Arquitectura .NET.
- Microsoft. (s.f.). *Cadenas de conexión (Entity Framework Core)*. Microsoft Learn.
- Microsoft. (s.f.). *Tipos de valor devuelto de acción de controlador de API web en ASP.NET Core*. Microsoft Learn.
- Microsoft. (s.f.). *Validación de modelos en ASP.NET Core MVC y Razor Pages*. Microsoft Learn.
- Richardson, C. (s.f.). *Pattern: Database per service*. Microservices.io.